



Term Project Final Report

**College of Engineering and Computer Science
COMP1020 Object-Oriented Programming & Data Structures
Spring 2026**

Team Name: Group 17

Project Title: Chess Plus

Team Members:

Nguyen Quy Tu ID: V202502163

Nguyen Dinh Nam Khanh ID: V202502768

Nguyen Thien Quang ID: V202502841

Bui Anh Minh ID: V202502451

Nguyen Minh Nghia ID: V202502131

1 Introduction and Project Overview

1.1 Problem Statement and Motivation

Chess is a highly complex two-player strategy game with an estimated 10^{120} possible game sequences. A digital chess engine must consistently enforce intricate rules while supporting efficient AI decision-making within practical response times. This makes chess an effective platform for demonstrating Object-Oriented Programming, data structures, and algorithmic problem solving in an interactive software system.

1.2 Project Objectives and Scope

The objective of *Chess Plus* is to develop a fully functional, rule-complete chess game in Java using Swing. The project implements all standard chess mechanics, including legal movement validation, check/checkmate detection, castling, pawn promotion, and Undo support. Both Human vs. Human and Human vs. Bot modes are supported.

The AI subsystem provides two difficulty levels: **BeginnerBot** (depth 3) and **AmateurBot** (depth 5), implemented using Minimax with Alpha-Beta pruning.

1.3 Key Features and Functionalities

The system provides an interactive 8×8 graphical chess board with drag-and-drop movement, legal move highlighting, algebraic move logging, and game-over dialogs. Advanced mechanics including pawn promotion and castling are fully implemented according to FIDE rules.

To maintain responsiveness during AI search, bot computation executes on a background thread while the UI renders from a temporary board snapshot to prevent flickering. Undo functionality supports both single-player and multiplayer restoration through stored **GameState** snapshots.

1.4 Modifications Since the Interim Stage

Since the interim report, the AI subsystem has been fully implemented using Minimax with Alpha-Beta pruning. The UI was upgraded from text-based rendering to graphical piece rendering, and algebraic move logging was added.

The project also introduced Undo support via the Memento design pattern, background-threaded bot computation, and the **ChessBot** interface for interchangeable AI difficulty levels. Additional utility abstractions such as **Move** and **MoveHelper** were added to improve modularity and centralize game-state operations.

2 System Requirements and Specifications

2.1 Core Functionalities

The system validates piece movement through piece-specific implementations of `isValidMove()`. Check detection and move legality are handled through `Board.isChecked()` and `willMoveResultInCheck()`, while checkmate and stalemate detection combine check validation with legal move generation.

Standard chess mechanics including castling, pawn promotion, Undo, and algebraic move logging are fully supported. The AI subsystem implements Minimax with Alpha-Beta pruning through the `BeginnerBot` and `AmateurBot` classes.

2.2 User Requirements and Expected Behaviors

At startup, players select either single-player or multiplayer mode, along with bot difficulty in single-player mode. Pieces respond to drag-and-drop mouse interactions, and invalid moves are automatically rejected.

Check states are displayed through `JOptionPane` alerts, while bot moves are executed automatically after a short artificial delay to improve visual feedback and gameplay clarity.

2.3 Performance, Usability, and Scalability

To maintain responsiveness during AI search, bot computation executes on a background `Thread`, while UI updates are safely dispatched through `SwingUtilities.invokeLater()`. Board snapshots are used during search to prevent rendering flicker caused by temporary simulation states.

The system also optimizes piece iteration through the `activePieceCoords[]` and `boardToIndex[]` structures, reducing board traversal from $O(64)$ to $O(n)$, where $n \leq 32$. Higher search depths improve AI strength at the cost of longer response times.

3 System Design and Architecture

3.1 High-Level Architecture and Module Organization

The system is divided into backend and frontend layers. The backend manages board state, move validation, check/checkmate detection, turn management, and AI computation, while the frontend handles board rendering, mouse interaction, and move submission. Communication between both layers is coordinated through `GameController`, and the UI never directly modifies the board state.

`ChessBoardUI` serves as the primary view and controller, owning the `Board`, `GameController`, and current `ChessBot` instance. The `Board` manages the `Square[8][8]` grid and active piece indices, while the `Piece` hierarchy encapsulates piece-specific movement logic. Shared utilities such as `MoveHelper` and `Move` support reusable game-state operations throughout the system.

3.2 OOP Concepts, Main Classes, and Interactions

The abstract class `Piece` forms the foundation of the piece hierarchy by storing shared properties such as colour and movement state. It defines the abstract method `isValidMove()`, which each concrete piece subclass overrides with its own movement rules. This demonstrates abstraction, inheritance, and runtime polymorphism, since the system can validate moves without knowing the concrete piece type in advance.

Each concrete subclass also implements `clonePiece()` for deep-copy snapshot generation during Undo restoration. Encapsulation is applied throughout the design. `Square` stores coordinates, board colour, and a nullable `Piece` reference through controlled access meth-

ods, while `Board` wraps the `Square[][]` structure and exposes only controlled operations for move execution and simulation.

`GameController` manages turn state and game settings, while `MoveHelper` centralises reusable logic for check detection, move simulation, legal move generation, and check-mate/stalemate testing. The `ChessBot` interface defines the contract for all AI implementations, allowing `BeginnerBot` and `AmateurBot` to be swapped at runtime without modifying the UI layer. `Move` encapsulates move-related data, while `GameState` stores board snapshots for the Undo system.

3.3 Class Summary

The system is organised into several major subsystems. The **core game logic subsystem** consists of `Board`, `Square`, and the `Piece` hierarchy, which together manage board state, movement validation, and check/checkmate detection.

The **AI subsystem** is built around the `ChessBot` interface, with `BeginnerBot` and `AmateurBot` implementing Minimax with Alpha-Beta pruning at different search depths.

Utility classes such as `MoveHelper`, `Move`, and `GameState` support reusable move validation, simulation, and Undo restoration.

The **user interface subsystem** includes `ChessBoardUI` and `MouseListener`, which together handle rendering and mouse interaction.

4 Data Structures and Algorithms

4.1 Selected Data Structures and Reasons

The system represents the chess board using a fixed-size `Square[8][8]` array, providing constant-time access to board cells while naturally mapping to chess coordinates.

To optimise check detection and move generation, the board maintains an `activePieceCoords[]` list containing only occupied positions, avoiding unnecessary full-board scans during move generation and check detection. The reverse-mapping array `boardToIndex[]` enables constant-time piece removal during captures.

Undo functionality is implemented through `Stack<GameState>`, which stores deep-copied board snapshots after each move. The AI subsystem uses `List<Move>` to dynamically store legal moves during Minimax search, while the UI caches piece images in a `Map<String, BufferedImage>` for efficient rendering.

4.2 Time and Space Complexity

Check detection in `isChecked()` operates in $O(n)$ time, where $n \leq 32$ is the number of active pieces, avoiding unnecessary full-board scans.

Legal move generation has worst-case complexity $O(n \times 64)$, since each active piece may evaluate all destination squares. Move simulation in `willMoveResultInCheck()` also runs in $O(n)$ time because the board is temporarily mutated before validation.

The Minimax AI algorithm has worst-case complexity $O(b^d)$, where b is the branching factor and d is the search depth. Since the average chess branching factor is approximately 35 moves per turn, naive Minimax becomes computationally expensive at higher depths.

Alpha-Beta pruning significantly reduces the number of evaluated states, approaching $O(b^{d/2})$ in favourable cases.

Active piece removal is optimised to $O(1)$ using the reverse-mapping structure, while Undo restoration requires $O(64)$ time to restore the complete board snapshot.

4.3 Algorithms Implemented

The AI subsystem uses the Minimax algorithm for two-player decision making. White acts as the maximising player while Black acts as the minimising player. The algorithm recursively generates legal moves to a configurable depth, evaluates terminal states using a static evaluation function, and propagates scores back through the game tree.

To improve search efficiency, Alpha-Beta pruning tracks the bounds α and β , pruning branches that cannot influence the final decision. This significantly reduces the number of evaluated positions and makes deeper searches practical.

Move ordering further improves pruning performance. Candidate moves are scored before evaluation, prioritising promotions and captures using the Most Valuable Victim / Least Valuable Attacker heuristic. Stronger moves are therefore explored earlier, leading to more pruning opportunities.

The evaluation function computes material balance by assigning weighted scores to each piece type and summing White and Black piece values.

4.4 Trade-offs and Optimisation Decisions

The evaluation function focuses only on material balance, which is computationally efficient but ignores positional concepts such as centre control and pawn structure. This trade-off allows deeper search depths while keeping response times practical.

The active piece list reduces unnecessary board traversal during move generation and check detection. To maintain UI stability during AI search, the system renders from a temporary board snapshot while the bot thread evaluates moves in the background.

The Minimax implementation uses a make/undo simulation pattern rather than cloning the entire board at each recursive node. Reusing a single board instance significantly reduces temporary object allocation and avoids the garbage-collection pauses observed in earlier implementations.

4.5 Undo via the Memento Pattern

Undo functionality is implemented using the Memento design pattern. After each committed move, the system stores a complete `GameState` snapshot containing deep copies of the board and pieces.

In single-player mode, Undo restores both the human move and the bot response by popping two states from the stack. In multiplayer mode, only one state is restored per action. Storing complete snapshots simplifies state restoration and guarantees correctness even for complex moves such as castling.

5 Implementation Details

5.1 Programming Language, Libraries, and Frameworks

The project is implemented entirely in Java (JDK 11+) without external libraries. The graphical interface uses Java Swing components including `JFrame`, `JPanel`, `JOptionPane`, `JScrollPane`, and `Graphics2D`. Piece images are loaded at startup using `ImageIO` from PNG assets stored locally.

Bot computation executes on a background `Thread`, while UI updates are safely dispatched through `SwingUtilities.invokeLater()` on the Event Dispatch Thread. The project does not require external build tools and can be compiled directly using `javac` and `java`.

5.2 File Structure and Package Organization

All source files are organised under the `src/lab/` package.

Core game management is handled by `Board.java`, `GameController.java`, and `Square.java`, which maintain board state, turn flow, and move validation. The chess piece hierarchy is built around the abstract `Piece.java` class, with subclasses such as `King.java`, `Queen.java`, `Rook.java`, `Bishop.java`, `Knight.java`, and `Pawn.java` implementing piece-specific movement behaviour.

Move simulation and Undo support are implemented through `Move.java`, `MoveHelper.java`, and `GameState`. The AI subsystem consists of `ChessBot.java`, `BeginnerBot.java`, and `AmateurBot.java`, which implement Minimax with Alpha-Beta pruning. User interface rendering and interaction are handled by `ChessBoardUI.java` and `MouseListener.java`.

The complete source code is publicly available on GitHub [8].

5.3 Key Implementation Decisions

The `isValidMove()` method is implemented as a pure predicate that validates movement rules without modifying board state. All state mutations are centralised through `Board.applyMove()` and `Board.undoMove()`, allowing AI search and check detection to simulate moves safely without corrupting the live board.

Drag-and-drop interaction is managed through `MouseListener`. When the user selects a piece, the system computes and highlights legal destination squares. During dragging, the selected piece is rendered at the cursor position, and when released the move is submitted to `tryMove()` for validation and execution.

Bot computation is executed on a background thread. Before search begins, the system captures a temporary board snapshot so the UI can continue rendering a stable state during Minimax simulation. Once the search completes, the selected move is delivered back to the Event Dispatch Thread through `SwingUtilities.invokeLater()`.

Castling validation is implemented in `King.isValidCastle()`, which verifies rook and king movement history, empty traversal squares, and check safety conditions. Pawn promotion is detected during move execution, with the bot automatically promoting to Queen while human players select a promotion piece through a dialog interface.

The Surrender feature reuses the same end-of-game flow as checkmate and stalemate through `promptNewGame()`, avoiding duplicated game-over logic. AI invocation is also centralised through `triggerBot()`, ensuring consistent behaviour after both normal moves and Undo restoration.

6 Testing and Evaluation

6.1 Test Cases and Testing Methodology

Testing was performed at both unit and system levels. `TestRunner.java` provides unit-style validation tests by constructing specific board states and directly evaluating `isValidMove()` results against expected outputs.

The complete system was also tested through manual gameplay sessions covering full match scenarios, including checkmate patterns such as Fool's Mate and Scholar's Mate, stalemate detection, castling, pawn promotion, Undo restoration, and both AI difficulty levels.

6.2 Backend Validation

Figure 1 shows representative console output from `TestRunner`, while Table 1 summarises the primary move validation and gameplay tests.

```
Set Pieces
[wR] [wK] [wB] [wQ] [wK] [wB] [wK] [wR]
[wP] [wP] [wP] [wP] [wP] [wP] [wP] [wP]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[ ] [ ] [ ] [ ] [ ] [ ] [ ] [ ]
[bP] [bP] [bP] [bP] [bP] [bP] [bP] [bP]
[bR] [bK] [bB] [bQ] [bK] [bB] [bK] [bR]

Move Tests
Pawn from (1,0) to (2,0): true
Pawn from (1,0) to (0,0): false
Rook from (0,0) to (3,0): false
Knight from (0,1) to (2,2): true
Bishop from (0,2) to (3,5): false
Rook from (7,7) to (7,4): false
```

Figure 1: Console output from `TestRunner` verifying move validation behaviour.

Table 1: Move validation test results

Test Case	Expected	Status
Pawn forward movement	Valid	Pass
Pawn backward movement	Invalid	Pass
Blocked rook movement	Invalid	Pass
Knight L-shape movement	Valid	Pass
Bishop diagonal movement	Valid	Pass
King moving into check	Rejected	Pass
Valid kingside castling	Allowed	Pass
Castling through check	Rejected	Pass
Checkmate detection	Detected	Pass
Stalemate detection	Detected	Pass
Pawn promotion	Success	Pass
Undo restoration (2-player)	Restored	Pass
Undo restoration (single-player)	Restored	Pass

6.3 Discussion of Correctness, Robustness, and Usability

All standard chess rules within the project scope were successfully validated through the test cases above. The Undo system restores complete board state safely because each `GameState` stores deep-copied snapshots through `clonePiece()` rather than references to live objects.

The UI remains responsive during AI computation due to the background threading model. Legal move highlighting improves usability by clearly indicating valid destinations, while the algebraic move log provides a complete history of gameplay actions.

6.4 Performance Evaluation

The Beginner bot at depth 3 consistently responds in under one second on tested positions. The Amateur bot at depth 5 typically responds within one to five seconds depending on board complexity.

No crashes, deadlocks, or invalid game-state transitions were observed across repeated gameplay sessions at both difficulty levels.

7 Challenges and Solutions

7.1 Technical Difficulties

One major technical challenge involved check detection during move simulation. Early implementations evaluated moves against the live board state rather than the temporary post-simulation state, producing incorrect results during deeper Minimax recursion. This was resolved through a strict apply-check-restore pattern inside `willMoveResultInCheck()`, where moves are temporarily applied, validated, and immediately reverted.

Another challenge was UI flickering during AI computation. Since the Minimax search mutates the same board object used by the renderer, temporary simulation states occasionally appeared on screen. The solution was to render from a temporary board snapshot

while `botThinking` is active, restoring normal rendering once the AI move is delivered through `SwingUtilities.invokeLater()`.

Castling validation introduced additional edge cases because FIDE rules require validation of movement history, path clearance, and check safety conditions simultaneously. These rules were implemented sequentially inside `King.isValidCastle()` to ensure correctness.

Managing the `activePieceCoords[]` structure also required careful optimisation. Captured pieces are removed using a swap-and-decrement strategy combined with the `boardToIndex[]` reverse map, allowing constant-time removal without shifting the entire array.

7.2 Design and Architectural Issues

`BeginnerBot` and `AmateurBot` currently share similar Minimax logic and differ mainly in search depth, resulting in partial code duplication. A future improvement would be to replace both classes with a single configurable implementation parameterised by depth.

Another architectural concern is that `ChessBoardUI` still handles both rendering and portions of gameplay flow, creating a risk of excessive responsibility concentration. This was partially mitigated by delegating reusable game-state logic to `MoveHelper` and turn management to `GameController`.

7.3 Lessons Learned

Frequent play-testing proved essential for detecting rule-validation bugs before they became deeply integrated into the codebase. The use of the `ChessBot` interface simplified runtime AI replacement and improved extensibility for future difficulty levels.

The project also reinforced the importance of thread-safe UI programming in Swing applications, since all interface updates must be dispatched through `SwingUtilities.invokeLater()` to avoid race conditions on the Event Dispatch Thread.

8 Conclusion and Future Work

Chess Plus successfully delivers a fully playable, rule-complete Java chess game with a Swing-based graphical interface, two AI difficulty levels, and major chess mechanics including castling, pawn promotion, check/checkmate detection, stalemate detection, Undo support, and algebraic move logging.

The project demonstrates the core Object-Oriented Programming principles of the course through the `Piece` hierarchy, which applies abstraction, inheritance, and runtime polymorphism. Encapsulation is enforced through the `Square` and `Board` classes, while the `ChessBot` interface enables interchangeable AI difficulty levels at runtime. The Memento pattern underpins the Undo system through `GameState` snapshots.

Beyond gameplay implementation, the project also required several architectural and optimisation decisions. The separation between backend logic and frontend rendering reduced coupling and simplified development, while background-threaded AI computation maintained UI responsiveness during Minimax search. The active piece index structure further improved move generation and check detection efficiency throughout gameplay.

Several limitations remain. En passant, repetition-based draws, and the 50-move rule are not currently implemented. The evaluation function considers only material balance and

does not include positional heuristics such as centre control or pawn structure. In addition, `BeginnerBot` and `AmateurBot` still contain partially duplicated Minimax logic differing mainly in search depth.

Future improvements include implementing the remaining chess rules, extending the evaluation function with positional analysis, refactoring the AI into a configurable `MinimaxBot(depth)` implementation, adding Quiescence Search and opening-book support, and introducing networked multiplayer and game timer functionality.

9 Team Contributions

Table 2 summarises each team member’s responsibilities throughout the project lifecycle.

Table 2: Team member contributions

Member	Contributions
Nguyen Quy Tu (Leader)	Project coordination and scheduling; implementation of <code>GameController</code> and <code>Board</code> ; check, checkmate, and stalemate detection; active piece index optimisation using <code>activePieceCoords[]</code> and <code>boardToIndex[]</code> .
Nguyen Dinh Nam Khanh	Implementation of <code>isValidMove()</code> across all piece subclasses; sliding-piece path validation; castling validation through <code>King.isValidCastle()</code> ; pawn promotion detection.
Nguyen Thien Quang	Development of <code>ChessBoardUI</code> and <code>MouseListener</code> ; drag-and-drop interaction; legal move highlighting; move log panel; bot threading and board snapshot rendering; Undo, Surrender, and New Game controls.
Bui Anh Minh	System integration and build assembly; development of the <code>TestRunner</code> test suite; debugging and integration-stage issue resolution.
Nguyen Minh Nghia	Implementation of the AI subsystem including <code>ChessBot</code> , <code>BeginnerBot</code> , and <code>AmateurBot</code> ; Minimax with Alpha-Beta pruning; MVV-LVA move ordering; evaluation function optimisation and object-churn reduction.

References

- [1] Knuth, D. E., & Moore, R. W. (1975). *An analysis of alpha-beta pruning*. Artificial Intelligence, 6(4), 293–326.
- [2] Oracle Corporation. (2024). *Java SE 11 API Documentation*. <https://docs.oracle.com/en/java/javase/11/>
- [3] FIDE. (2024). *FIDE Laws of Chess*. <https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
- [4] Chessprogramming Wiki. (2024). *Move Ordering*. https://www.chessprogramming.org/Move_Ordering

- [5] Chessprogramming Wiki. (2024). *Minimax*. <https://www.chessprogramming.org/Minimax>
- [6] Chessprogramming Wiki. (2024). *Alpha-Beta*. <https://www.chessprogramming.org/Alpha-Beta>
- [7] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [8] Group 17. (2026). *Chess Plus - Source Code Repository*. <https://github.com/EnTeeKiu/Chess>

A Additional Screenshots

Figure 2 shows the *Chess Plus* graphical interface during gameplay. The interface includes PNG-based piece rendering, drag-and-drop interaction, legal move highlighting, algebraic move logging, and the Undo, New Game, and Surrender controls.

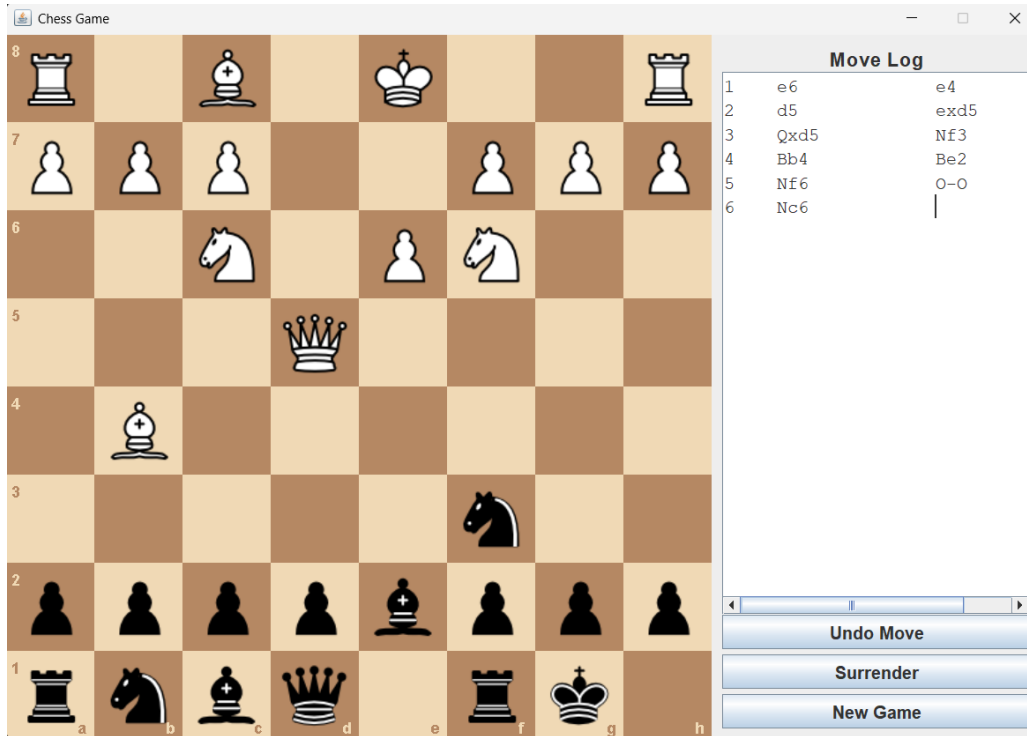


Figure 2: *Chess Plus* graphical interface during an active game session.

B Extended Test Cases

Additional validation was performed through `TestRunner.java` using manually constructed board states. These tests covered edge cases including invalid castling through attacked squares, self-check prevention, Undo restoration after bot moves, and pawn promotion behaviour.

The complete console output from representative **TestRunner** sessions is shown in Figure 1 in Section 6.

C Class Diagram (Full)

Figure 3 presents the complete UML class diagram for the *Chess Plus* system. The diagram summarises the relationships between the *Piece* hierarchy, game-state management classes, AI subsystem, and user interface components discussed in Section 3.

D Group Project Task Assignment and Contribution Agreement

uml_full.png

Figure 3: Complete UML class diagram of the *Chess Plus* system.

Group Members Information

No.	Full Name	Student ID	Email
1	Nguyen Quy Tu (Leader)	V202502163	25tu.nq@vinuni.edu.vn
2	Nguyen Dinh Nam Khanh	V202502768	25khanh.ndn@vinuni.edu.vn
3	Nguyen Thien Quang	V202502841	25quang.nt@vinuni.edu.vn
4	Bui Anh Minh	V202502451	25minh.ba@vinuni.edu.vn
5	Nguyen Minh Nghia	V202502131	25nghia.nm@vinuni.edu.vn

Task Progress and Completion Status

Task	Member	Status	% Done
Core framework and board state	Nguyen Quy Tu	Completed	100%
Piece movement and rule validation	Nguyen Dinh Nam Khanh	Completed	100%
Graphical interface and interaction	Nguyen Thien Quang	Completed	100%
System integration and testing	Bui Anh Minh	Completed	100%
AI subsystem and Minimax search	Nguyen Minh Nghia	Completed	100%

Contribution Agreement

Member Name	Participation	Agree	Comments
Nguyen Quy Tu	100%	Yes	
Nguyen Dinh Nam Khanh	100%	Yes	
Nguyen Thien Quang	100%	Yes	
Bui Anh Minh	100%	Yes	
Nguyen Minh Nghia	100%	Yes	

Declaration

We certify that all contributions and reported responsibilities accurately reflect the work completed throughout the project lifecycle.

Member Name	Signature	Date
Nguyen Quy Tu	Tu	02/06/2026
Nguyen Dinh Nam Khanh	Khanh	02/06/2026
Nguyen Thien Quang	Quang	02/06/2026
Bui Anh Minh	Minh	02/06/2026
Nguyen Minh Nghia	Nghia	02/06/2026

For TA Use Only:

Approved by: _____ Date: _____